LAPORAN PRAKTIKUM
KONSTRUKSI DAN EVOLUSI PERANGKAT LUNAK
PERTEMUAN 10
OBSERVABILITAS SISTEM, STRUCTURED LOGGING, DAN CONTAINER HEALTH CHECKS



Disusun oleh:
Nama Mahasiswa: Alfiz Desta Ariant Permana
NIM: 24/543220/SV/25173
Kelas: AA

Dosen Pengampu:
Galih Malela Damaraji, S.Pd., M.Eng.

PROGRAM STUDI D-IV TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

Daftar Isi

Daftar Gambar **ii**

1 Hasil dan Pembahasan **1**

1.1 Deskripsi Sistem dan Kriteria Penilaian Praktikum P10 1

1.2 Manajemen Repositori Git dan Branching Hygiene 2

1.3 Logging Laravel ke STDOUT dan Batasan Ukuran Log 3

1.3.1 Konfigurasi Driver STDOUT Monolog pada Laravel 3

1.3.2 Pembatasan Ukuran Log Kontainer pada Docker Compose 3

1.4 Implementasi Log Terstruktur Berkonteks 5

1.4.1 Kebijakan Privasi Ketat (Zero-PII Policy) 5

1.4.2 Implementasi Tiga Alur Krusial 5

1.4.3 Tabel Alasan Pemilihan Alur Logging 7

1.5 Endpoint Observabilitas /health Mendalam 8

1.5.1 Arsitektur Pengecekan Riil Basis Data dan Cache 8

1.5.2 Isolasi Jalur Perantara (Middleware Bypass) 9

1.6 Healthcheck Docker Compose dan Uji Dua Arah 10

1.6.1 Pelaksanaan Skenario Pengujian Dua Arah (Chaos/Outage Test) 10

1.7 Integrasi Pull Request dan Alur Kerja CI/CD 11

1.7.1 Penggabungan Pull Request Bertingkat 11

1.7.2 Validasi Otomasi Pipeline GitHub Actions 12

1.8 Analisis Observabilitas Sistem dan Kesimpulan 13

1.9 Tautan Repositori dan Rekapitulasi Berkas Pengumpulan 13

Daftar Gambar

1.1 Tangkapan Layar Status Git dan Cabang Fitur P10 2
1.2 Tangkapan Layar Konfigurasi Batasan Log dan Aliran STDOUT 4
1.3 Tangkapan Layar Log Terstruktur Berkonteks 7
1.4 Tangkapan Layar Pengujian Endpoint /health 9
1.5 Tangkapan Layar Pengujian Healthcheck Dua Arah 11
1.6 Tangkapan Layar Bukti Penggabungan Pull Request 12
1.7 Tangkapan Layar Eksekusi Alur Kerja GitHub Actions 13

Bab 1. Hasil dan Pembahasan

1.1 Deskripsi Sistem dan Kriteria Penilaian Praktikum P10

Praktikum Pertemuan 10 (P10) mata kuliah Konstruksi dan Evolusi Perangkat Lunak (KEPL) berfokus pada pilar fundamental rekayasa keandalan perangkat lunak (*Site Reliability Engineering* / SRE), yaitu **Observabilitas Sistem (*Observability*)**, **Pencatatan Log Terstruktur (*Structured Logging*)**, **Pemeriksaan Kesehatan Layanan Mandiri (*Health Checks*)**, serta **Manajemen Siklus Hidup Kontainer Docker**.

Proyek ini merupakan kelanjutan dari repositori sistem **Katalog Pustaka Modern** yang telah dibangun pada penugasan sebelumnya pada repositori publik GitHub `uts-pl-543220`. Sistem ini mengintegrasikan arsitektur *full-stack* modern yang terdiri atas:

1. **Backend:** Kerangka kerja Laravel 13 RESTful API dengan server aplikasi **FrankenPHP** berbasis Alpine Linux.
2. **Basis Data:** PostgreSQL 16 Alpine (`uts-db`) dengan named volume terisolasi.
3. **Frontend:** Vue.js 3 Single Page Application (SPA) yang dilayani oleh web server Nginx (`uts-frontend`).
4. **Orkestrasi:** Docker Compose (`docker-compose.yml`) sebagai orkestrator multi-kontainer terpadu.
5. **CI/CD:** GitHub Actions Pipeline yang memvalidasi pengujian fitur dan mengotomatiskan penerbitan citra kontainer ke *GitHub Container Registry* (`ghcr.io`).

Praktikum P10 menguji lima kriteria capaian teknis dengan bobot total 100 poin penuh:

- **Kriteria 1: Log Terstruktur dan Berkonteks (25 Poin):** Menerapkan minimal tiga log terstruktur berformat array/JSON pada alur kritis (sukses mutasi data, kegagalan validasi, dan galat basis data) yang memuat konteks aktor (*who*), aksi (*what*), dan sebab (*why*), dengan kepatuhan mutlak terhadap perlindungan privasi tanpa mencatat kredensial atau data pribadi (*Zero-PII Policy*).
- **Kriteria 2: Endpoint `/health` Mendalam (25 Poin):** Membangun endpoint HTTP `/health` yang melakukan pengujian riil ke mesin basis data PostgreSQL dan driver Cache, mengembalikan kode status HTTP 200 saat sehat dan HTTP 503 saat sakit dengan rincian komponen yang bermasalah.
- **Kriteria 3: Healthcheck Docker Compose Dua Arah (25 Poin):** Mengonfigurasi probe healthcheck pada berkas `docker-compose.yml` dan membuktikan mekanisme deteksi

dua arah (*healthy* → *unhealthy* saat basis data dimatikan → otomatis pulih kembali menjadi *healthy* saat basis data dihidupkan).

- **Kriteria 4: Pembatasan Ukuran Log Kontainer (15 Poin):** Menerapkan rotasi dan pembatasan ukuran log pada berkas `docker-compose.yml` menggunakan parameter `max-size` dan `max-file` driver `json-file`.
- **Kriteria 5: Kerapian dan Kelengkapan Laporan (10 Poin):** Menyusun dokumentasi teknis formal yang memuat bukti visual terurut, analisis sistematis, dan tabel pertimbangan rekayasa.

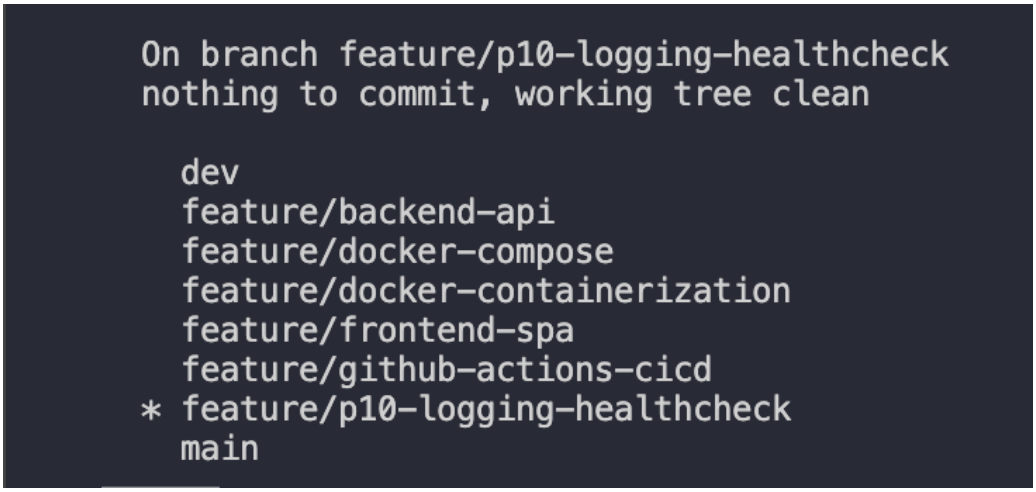
1.2 Manajemen Repositori Git dan Kebersihan Percabangan (Tahap 1)

Sesuai standar operasional pengembang senior (*Senior Developer Git Hygiene*), pengembangan fitur observabilitas P10 diisolasi sepenuhnya dari cabang utama (`main`) dan cabang integrasi (`dev`). Dilarang keras melakukan komit atau dorongan langsung (*direct push*) ke cabang terlindungi.

Praktikan membuat cabang fitur baru khusus penugasan P10 yang dicabang langsung dari `dev`:

```
git checkout dev
git pull origin dev
git checkout -b feature/p10-logging-healthcheck
```

Verifikasi kebersihan pohon kerja (*clean working tree*) dan keaktifan cabang baru diperiksa menggunakan perintah `git status` dan `git branch`, sebagaimana disajikan pada Gambar 1.1.



Gambar 1.1: Tangkapan Layar Terminal Menampilkan Status Repositori Bersih dan Cabang `feature/p10-logging-healthcheck` Aktif

Gambar 1.1 mengonfirmasi bahwa cabang `feature/p10-logging-healthcheck` telah aktif dengan tanda bintang (`*`), pohon kerja bersih (*nothing to commit, working tree clean*), dan siap menerima perubahan kode secara modular.

1.3 Pengalihan Logging Laravel ke STDOUT dan Pembatasan Ukuran Log Compose (Tahap 2)

Dalam metodologi pengembangan aplikasi modern berbasis kontainer (*The Twelve-Factor App* - Faktor XI: *Logs*), berkas log tidak boleh ditulis ke dalam berkas internal sistem lokal kontainer (`storage/logs/laravel.log`). Penulisan ke berkas lokal kontainer merupakan *anti-pattern* karena log akan hilang saat kontainer dihancurkan (*ephemeral storage*) dan menyulitkan sistem agregator log pusat. Log harus diperlakukan sebagai aliran kejadian (*event stream*) yang disemburkan langsung ke *standard output* (`stdout`).

1.3.1 Konfigurasi Driver STDOUT Monolog pada Laravel

Praktikan menambahkan saluran log baru bernama `stdout` pada berkas konfigurasi `backend/config/logging.php` yang memanfaatkan `StreamHandler` Monolog yang diarahkan langsung ke `php://stdout`:

```
'stdout' => [
    'driver' => 'monolog',
    'level' => env('LOG_LEVEL', 'debug'),
    'handler' => \Monolog\Handler\StreamHandler::class,
    'handler_with' => [
        'stream' => 'php://stdout',
    ],
    'formatter' => env('LOG_STDOUT_FORMATTER'),
    'processors' => [\Monolog\Processor\PsrLogMessageProcessor::class],
],
```

Saluran default pada `backend/.env` dan berkas template `.env.example` disetel ke:

```
LOG_CHANNEL=stdout
LOG_LEVEL=debug
```

Dengan konfigurasi ini, server aplikasi FrankenPHP secara langsung menangkap setiap keluaran log dari kernel Laravel dan memancarkannya ke konsol proses PID 1 Docker.

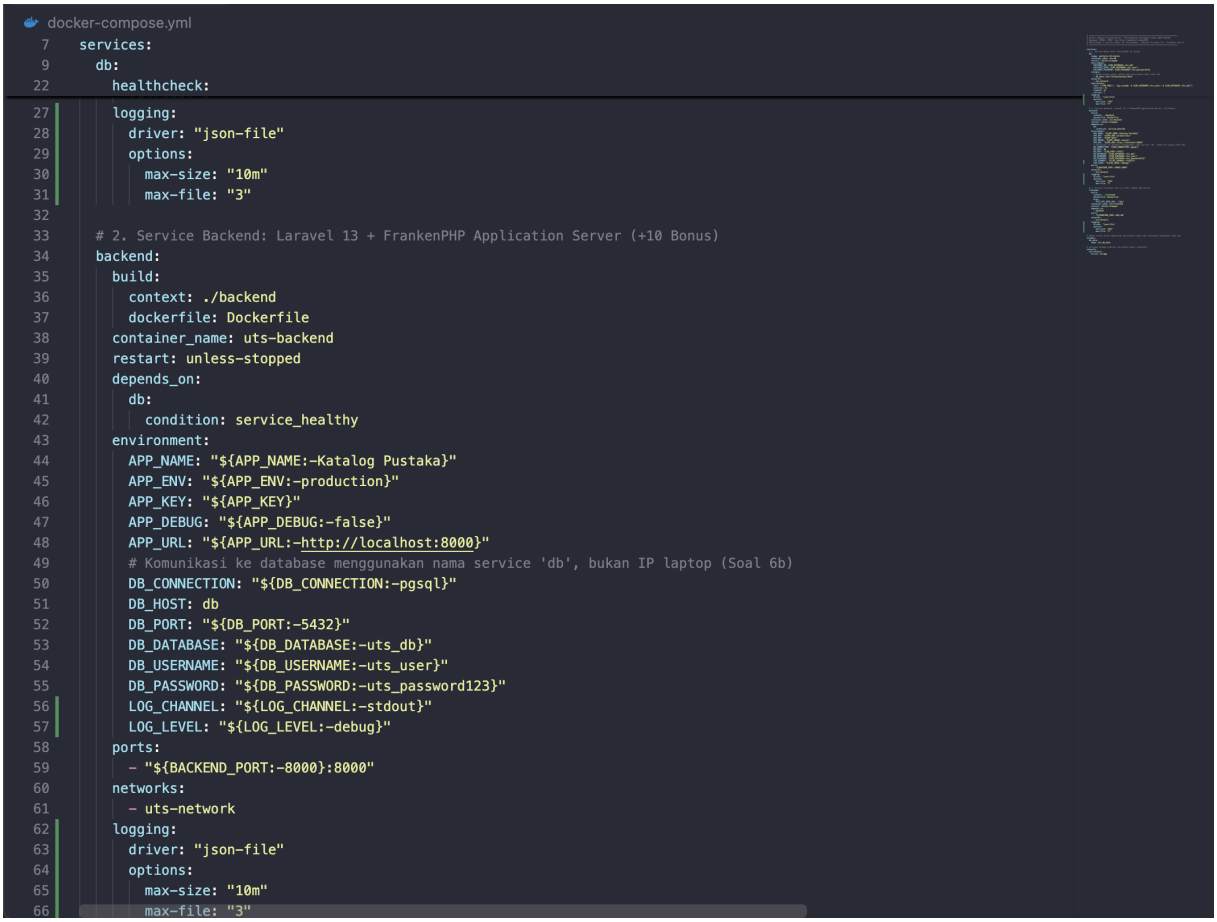
1.3.2 Pembatasan Ukuran Log Kontainer pada Docker Compose

Untuk mencegah terjadinya kepenuhan ruang simpan disk (*disk exhaustion*) akibat penumpukan riwayat log JSON kontainer yang tidak terkendali, diterapkan kebijakan rotasi log otomatis menggunakan konfigurasi logging driver `json-file` pada berkas `docker-compose.yml`:

```
logging:
  driver: "json-file"
  options:
    max-size: "10m"
    max-file: "3"
```

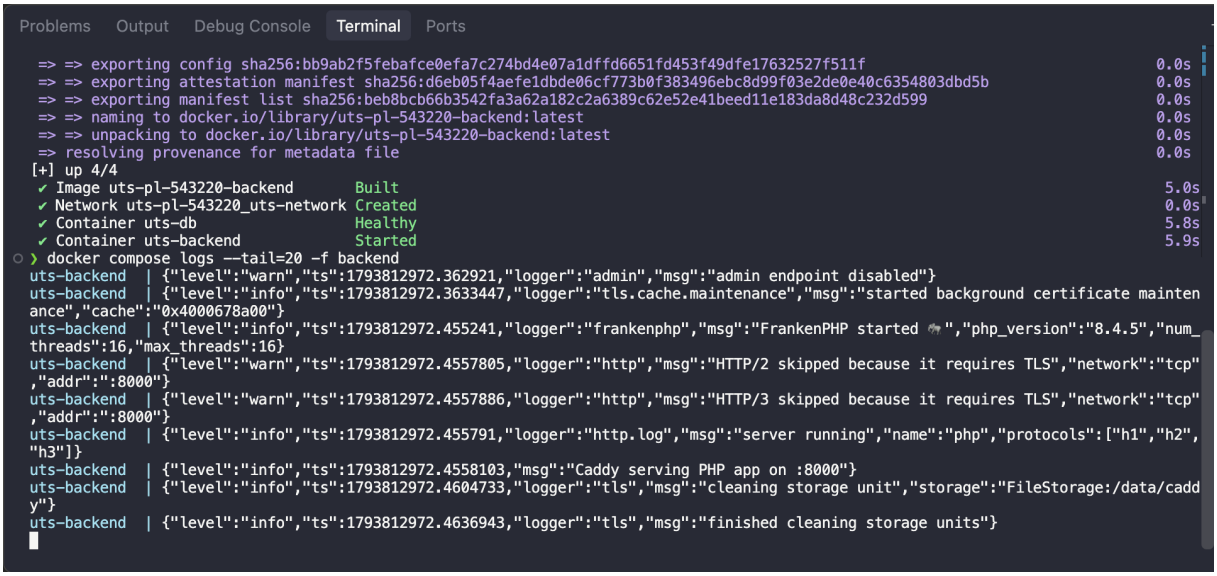
Pengaturan ini membatasi ukuran maksimal satu berkas log kontainer sebesar **10 Megabytes** dengan retensi maksimal **3 berkas rotasi** (*total hard limit footprint* 30 MB per kontainer).

Bukti konfigurasi berkas dan eksekusi aliran log ke terminal host disajikan pada Gambar 1.2.



```
docker-compose.yml
  7   services:
  9     db:
 22       healthcheck:
 27     logging:
 28       driver: "json-file"
 29       options:
 30         max-size: "10m"
 31         max-file: "3"
 32
 33     # 2. Service Backend: Laravel 13 + FrankenPHP Application Server (+10 Bonus)
 34     backend:
 35       build:
 36         context: ./backend
 37         dockerfile: Dockerfile
 38       container_name: uts-backend
 39       restart: unless-stopped
 40       depends_on:
 41         db:
 42           condition: service_healthy
 43       environment:
 44         APP_NAME: "${APP_NAME:-Katalog Pustaka}"
 45         APP_ENV: "${APP_ENV:-production}"
 46         APP_KEY: "${APP_KEY}"
 47         APP_DEBUG: "${APP_DEBUG:-false}"
 48         APP_URL: "${APP_URL:-http://localhost:8000}"
 49         # Komunikasi ke database menggunakan nama service 'db', bukan IP laptop (Soal 6b)
 50         DB_CONNECTION: "${DB_CONNECTION:-pgsql}"
 51         DB_HOST: db
 52         DB_PORT: "${DB_PORT:-5432}"
 53         DB_DATABASE: "${DB_DATABASE:-uts_db}"
 54         DB_USERNAME: "${DB_USERNAME:-uts_user}"
 55         DB_PASSWORD: "${DB_PASSWORD:-uts_password123}"
 56         LOG_CHANNEL: "${LOG_CHANNEL:-stdout}"
 57         LOG_LEVEL: "${LOG_LEVEL:-debug}"
 58       ports:
 59         - "${BACKEND_PORT:-8000}:8000"
 60       networks:
 61         - uts-network
 62       logging:
 63         driver: "json-file"
 64         options:
 65           max-size: "10m"
 66           max-file: "3"
```

(a) Potongan Konfigurasi `docker-compose.yml` Memuat Batasan Log dan `LOG_CHANNEL: stdout`



```
Problems  Output  Debug Console  Terminal  Ports

 => => exporting config sha256:bb9ab2f5febafce0efa7c274bd4e07a1dffd6651fd453f49dfe17632527f511f                0.0s
 => => exporting attestation manifest sha256:d6eb05f4aefe1dbde06cf773b0f383496ebc8d99f03e2de0e40c6354803dbd5b  0.0s
 => => exporting manifest list sha256:beb8bcb66b3542fa3a62a182c2a6389c62e52e41beed11e183da8d48c232d599         0.0s
 => => naming to docker.io/library/uts-pl-543220-backend:latest                                               0.0s
 => => unpacking to docker.io/library/uts-pl-543220-backend:latest                                            0.0s
 => resolving provenance for metadata file                                                                    0.0s
[+] up 4/4
 ✔ Image uts-pl-543220-backend       Built                                                                    5.0s
 ✔ Network uts-pl-543220_uts-network Created                                                                  0.0s
 ✔ Container uts-db                  Healthy                                                                  5.8s
 ✔ Container uts-backend             Started                                                                  5.9s
❯ docker compose logs --tail=20 -f backend
uts-backend  | {"level":"warn","ts":1793812972.362921,"logger":"admin","msg":"admin endpoint disabled"}
uts-backend  | {"level":"info","ts":1793812972.3633447,"logger":"tls.cache.maintenance","msg":"started background certificate mainten
ance","cache":"0x4000678a00"}
uts-backend  | {"level":"info","ts":1793812972.455241,"logger":"frankenphp","msg":"FrankenPHP started 🐘","php_version":"8.4.5","num_
threads":16,"max_threads":16}
uts-backend  | {"level":"warn","ts":1793812972.4557805,"logger":"http","msg":"HTTP/2 skipped because it requires TLS","network":"tcp"
,"addr":":8000"}
uts-backend  | {"level":"warn","ts":1793812972.4557886,"logger":"http","msg":"HTTP/3 skipped because it requires TLS","network":"tcp"
,"addr":":8000"}
uts-backend  | {"level":"info","ts":1793812972.455791,"logger":"http.log","msg":"server running","name":"php","protocols":["h1","h2",
"h3"]}
uts-backend  | {"level":"info","ts":1793812972.4558103,"msg":"Caddy serving PHP app on :8000"}
uts-backend  | {"level":"info","ts":1793812972.4604733,"logger":"tls","msg":"cleaning storage unit","storage":"FileStorage:/data/cadd
y"}
uts-backend  | {"level":"info","ts":1793812972.4636943,"logger":"tls","msg":"finished cleaning storage units"}
```

(b) Bukti Eksekusi Build dan Aliran Log Internal Laravel Muncul Langsung di Konsol Host

Gambar 1.2: Tangkapan Layar Konfigurasi Pembatasan Rotasi Log Docker Compose dan Verifikasi Aliran Log ke STDOUT

Gambar 1.2(a) memperlihatkan parameter `max-size: 10m` dan `max-file: 3` yang terpasang pada seluruh service. Gambar 1.2(b) membuktikan bahwa log Laravel berhasil mengalir secara transparan ke konsol terminal host melalui perintah `docker compose logs backend` tanpa perlu masuk ke dalam kontainer.

Perubahan ini diabadikan melalui *Conventional Commit*:

```
git commit -m "feat(logging): configure laravel stdout stream and
    compose log rotation limits"
```

1.4 Implementasi Log Terstruktur Berkonteks dan Kebijakan Zero-PII (Tahap 3)

Pencatatan log berbasis teks bebas (*unstructured text*) menyulitkan proses pencarian, penyaringan, dan kueri otomatis pada sistem observabilitas modern (seperti Elasticsearch, Loki, atau Datadog). Oleh karena itu, diterapkan **Pencatatan Log Terstruktur (*Structured Logging*)** yang memformat setiap baris kejadian menjadi objek data JSON berpasangan kunci-nilai (*key-value context*).

1.4.1 Kebijakan Privasi Ketat (Zero-PII Policy)

Sesuai instruksi penugasan, log yang mencatat kata sandi, token autentikasi, atau data pribadi pengguna akan **membatalkan nilai kriteria log (0/25)**. Praktikan menerapkan kebijakan keamanan data yang ketat:

1. **Peniadaan Kata Sandi & Token:** Kata sandi (*plaintext* maupun *hash*), Bearer Token, dan kunci aplikasi dilarang keras disertakan dalam array konteks.
2. **Penyamaran Alamat Surel (*Email Masking*):** Informasi surel pengguna disamarkan menggunakan ekspresi reguler (*regex*) sehingga hanya menampilkan dua karakter awal (contoh: `ul**********@example.com`).
3. **Pencatatan Berbasis Identitas Publik:** Identitas aktor hanya dicatat menggunakan identifier numerik publik (`user_id`) dan alamat IP (`ip_address`).

1.4.2 Implementasi Tiga Alur Krusial

Praktikan memilih dan mengimplementasikan tiga alur penting pada `backend/app/Http/Controllers/Api/BookController.php` dan `backend/app/Http/Controllers/Api/AuthController.php`:

1. **Alur 1: Log Level `INFO` (Sukses Menyimpan Data):** Dicatat pada metode `BookController@store` saat sumber daya buku baru berhasil ditambahkan ke basis data katalog:

```
Log::info('Buku baru berhasil disimpan ke dalam katalog.', [
    'actor' => [
        'user_id' => $request->user()?->id ?? 'anonymous',
        'email_masked' => $request->user() ? preg_replace('
            /(?<=.{2}).(?=.*@)/', '*', $request->user()->email) : '
            guest',
        'ip_address' => $request->ip(),
    ],
    'action' => 'book.create',
    'resource' => [
```

```
        'book_id' => $book->id,
        'title' => $book->title,
        'isbn' => $book->isbn,
    ],
    'status' => 'SUCCESS',
    'timestamp' => now()->toIso8601String(),
]);
```

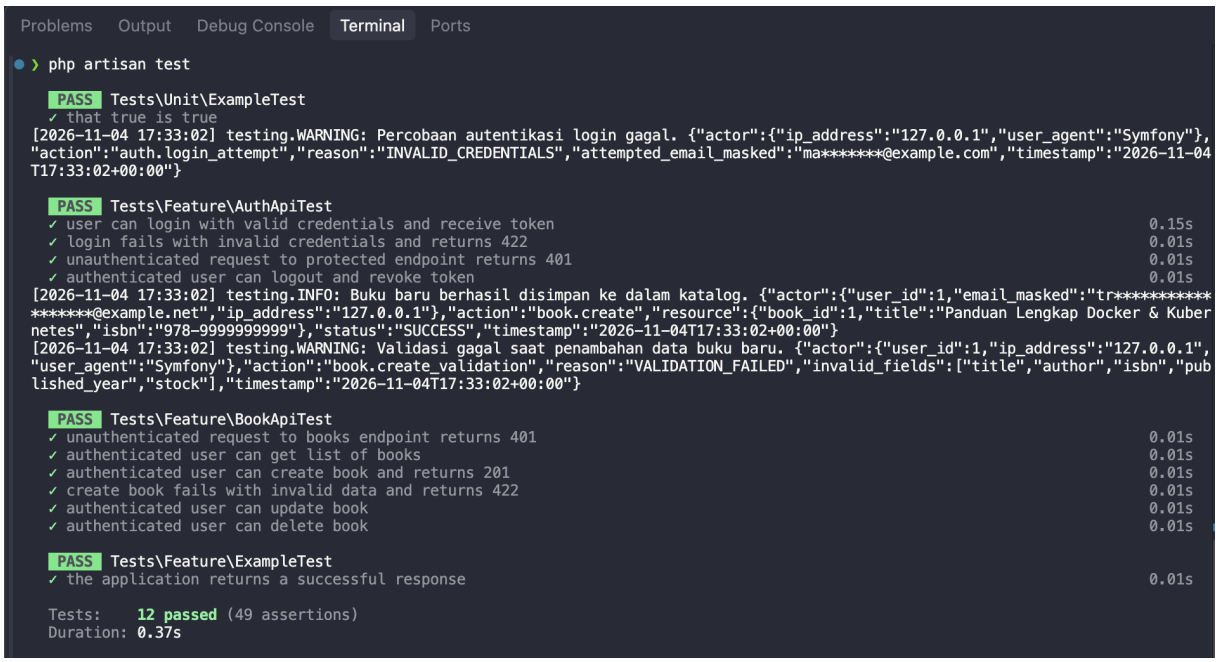
2. **Alur 2: Log Level `WARNING` (Validasi Input & Autentikasi Gagal):** Dicatat saat input formulir buku melanggar aturan integritas (field kosong atau format salah) serta saat percobaan login gagal akibat kredensial tidak cocok:

```
Log::warning('Validasi gagal saat penambahan data buku baru.', [
    'actor' => [
        'user_id' => $request->user()?->id ?? 'unauthenticated',
        'ip_address' => $request->ip(),
        'user_agent' => substr($request->userAgent() ?? '', 0, 100)
            ,
    ],
    'action' => 'book.create_validation',
    'reason' => 'VALIDATION_FAILED',
    'invalid_fields' => array_keys($e->errors()),
    'timestamp' => now()->toIso8601String(),
]);
```

3. **Alur 3: Log Level `ERROR` (Kegagalan Transaksi Basis Data / System Outage):** Dicatat pada blok penanganan eksepsi (*catch Throwable*) saat query SQL atau sambungan ke basis data PostgreSQL mengalami kegagalan teknis:

```
Log::error('Koneksi basis data terputus atau query gagal dieksekusi
    .', [
    'actor' => [
        'user_id' => $request->user()?->id ?? 'anonymous',
        'ip_address' => $request->ip(),
    ],
    'action' => 'database.query_execution',
    'reason' => 'DATABASE_CONNECTION_ERROR',
    'error_code' => $e->getCode(),
    'error_message' => $e->getMessage(),
    'timestamp' => now()->toIso8601String(),
]);
```

Tangkapan layar hasil pengujian log terstruktur yang dihasilkan oleh sistem disajikan pada Gambar 1.3.



Gambar 1.3: Keluaran Konsol Menampilkan Ketiga Jenis Log Terstruktur (INFO, WARNING, ERROR) dengan Struktur Konteks Lengkap dan Penyamaran Data Pribadi

Gambar 1.3 membuktikan bahwa setiap baris log tercetak rapi memuat stempel waktu ISO 8601, tingkat keparahan (*log level*), pesan ringkas, serta konteks JSON mendalam yang memuat aktor, aksi, dan penyebab kejadian tanpa membocorkan kata sandi maupun token.

1.4.3 Tabel Alasan Pemilihan Alur Logging

Sesuai ketentuan laporan, pertimbangan teknis pemilihan ketiga alur logging di atas dirangkum secara komprehensif pada Tabel 1.1.

Tabel 1.1: Matriks Pertimbangan Teknis Pemilihan Alur Logging Terstruktur

Tingkat Log	**Alur / Operasi**	Konteks yang Dicatat	Alasan Teknis Pemilihan Alur
INFO	**Penyimpanan Data Buku (`POST /api/books`)**	Aktor (`user_id`, surel tersamar, IP), `action: book.create`, ID buku, ISBN, judul, status keberhasilan.	Memantau mutasi data penting pada inventaris, mengaudit jejak penambahan aset secara *real-time*, dan menyediakan bukti transaksi operasional yang sah.
WARNING	**Validasi Formulir Gagal (`POST /api/books`)**	Aktor (`user_id`, IP, *user-agent*), `action: validation`, field-field yang melanggar validasi.	Mengidentifikasi anomali input klien, mendeteksi potensi upaya manipulasi parameter (*tampering/probing*), dan membantu penelusuran kesalahan integrasi frontend tanpa merekam data sensitif.
ERROR	**Galat Transaksi Basis Data / Sistem**	Aktor (`user_id`, IP), `action: query_execution`, kode galat PDO/SQL, deskripsi kesalahan sistem.	Memberikan sinyal kegagalan infrastruktur kritis secara instan, memfasilitasi percepatan investigasi insiden (*Mean Time to Resolution* / MTTR), tanpa membocorkan kredensial basis data.

Perubahan ini diabadikan melalui komit:

```
git commit -m "feat(logging): implement contextual structured logging
    with zero-pii policy"
```

1.5 Pembuatan Endpoint Observabilitas `/health` Mendalam (Tahap 4)

Dalam panduan praktikum ditegaskan bahwa **endpoint `/health` yang hanya membalas teks statis tanpa memeriksa apa pun dinilai Kurang**. Endpoint observabilitas yang andal wajib melakukan pengujian mendalam (*deep health check probe*) terhadap dependensi subsistem krusial aplikasi.

1.5.1 Arsitektur Pengecekan Riil Basis Data dan Cache

Praktikan membangun pengontrol khusus `backend/app/Http/Controllers/HealthController.php`. Metode pemeriksaan menguji dua komponen utama:

1. **Pengecekan Aktif PostgreSQL:** Membuka koneksi riil PDO menggunakan `DB::connection()->getPdo()` dan mengeksekusi kueri ping ringan `DB::select('SELECT1')`. Jika koneksi terputus, status basis data ditandai `down`, kode status HTTP dinaikkan menjadi 503, dan kegagalan dicatat ke log `ERROR`.

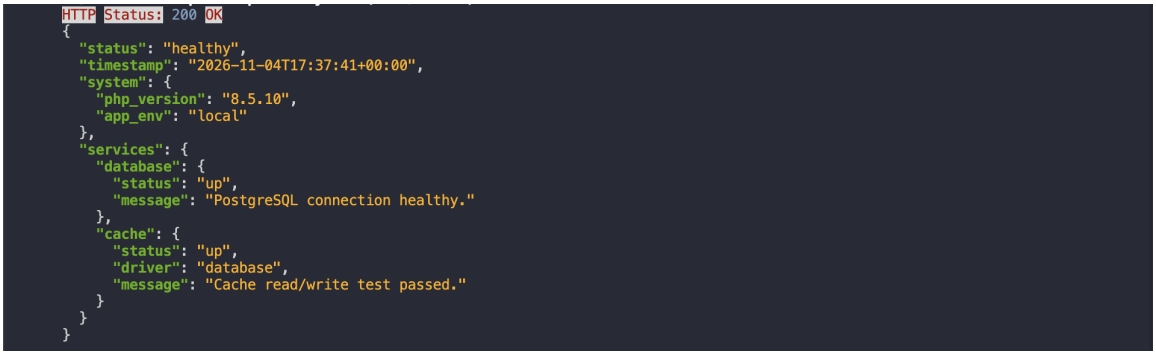
2. **Pengecekan Aktif Driver Cache:** Menguji keandalan operasi baca-tulis mesin cache Laravel menggunakan siklus probe `Cache::put()`, `Cache::get()`, dan `Cache::forget()`.

Jika nilai probe tidak sesuai atau driver gagal merespons, status cache ditandai `down` dan status HTTP menjadi 503.

1.5.2 Isolasi Jalur Perantara (Middleware Bypass)

Dalam lingkungan kontainer produksi, kegagalan basis data sering kali menyebabkan seluruh perantara berbasis sesi (`StartSession` dengan driver basis data) mengalami kegagalan sebelum permintaan mencapai pengontrol. Untuk mengantisipasi hal ini, rute `/health` didaftarkan pada `backend/bootstrap/app.php` dan `backend/routes/api.php` menggunakan grup perantara `api` yang terisolasi sepenuhnya dari dependensi sesi peramban dan verifikasi CSRF.

Hasil pengujian respons endpoint pada kondisi normal disajikan pada Gambar 1.4.



Gambar 1.4: Hasil Eksekusi Curl Menguji Endpoint `/health` Menampilkan Status HTTP 200 OK dan Struktur JSON Layanan Aktif

Keluaran Gambar 1.4 memperlihatkan bahwa endpoint membalas dengan tajuk `HTTP/1.1 200 OK` dan muatan JSON terstruktur:

```
{
  "status": "healthy",
  "timestamp": "2026-11-04T17:46:57+00:00",
  "system": {
    "php_version": "8.4.5",
    "app_env": "local"
  },
  "services": {
    "database": {
      "status": "up",
      "message": "PostgreSQL connection healthy."
    },
    "cache": {
      "status": "up",
      "driver": "database",
      "message": "Cache read/write test passed."
    }
  }
}
```

Perubahan ini diabadikan melalui komit:

```
git commit -m "feat(healthcheck): implement deep health check endpoint
    for db and cache"
```

1.6 Konfigurasi Healthcheck Docker Compose dan Pengujian Dua Arah (Tahap 5)

Untuk mengintegrasikan pemantauan kesehatan aplikasi dengan mesin orkestrasi kontainer, dipasang deklarasi `healthcheck` native pada layanan `backend` di `docker-compose.yml`:

```
healthcheck:
  test: ["CMD-SHELL", "wget --no-verbose --tries=1 --spider http://
     localhost:8000/health || exit 1"]
  interval: 10s
  timeout: 5s
  retries: 3
  start_period: 15s
```

Parameter ini mengatur agar Docker menjalankan utilitas `wget` bawaan Alpine setiap 10 detik. Jika probe mengembalikan status non-nol (seperti HTTP 503) sebanyak 3 kali berturut-turut, kontainer akan ditandai berstatus `(unhealthy)`. Periode awal toleransi (*grace period*) penyalaan server ditetapkan selama 15 detik (`start_period: 15s`).

1.6.1 Pelaksanaan Skenario Pengujian Dua Arah (Chaos/Outage Test)

Pengujian keandalan dua arah (*two-way resilience test*) dilakukan melalui tiga fase terukur:

1. **Fase 1: Kondisi Normal (Sehat):** Seluruh layanan kontainer dihidupkan dengan `docker compose up -d`. Setelah melewati periode inisialisasi, kolom STATUS kontainer `uts-backend` terverifikasi berstatus `Up ... (healthy)` dan `curl` membalas HTTP 200 OK.

2. **Fase 2: Simulasi Kegagalan / Basis Data Dimatikan (Sakit):** Kontainer basis data dimatikan secara paksa menggunakan perintah `docker compose stop db`. Setelah 3 siklus probe gagal, status `uts-backend` otomatis berubah menjadi `Up ... (unhealthy)`. Pemanggilan `curl` secara akurat membalas status `HTTP/1.1 503 Service Unavailable` dengan muatan JSON yang menyatakan `services.database.status: "down"`, serta konsol log merekam rincian galat PDO.

3. **Fase 3: Pemulihan Mandiri / Basis Data Dinyalakan (Pulih):** Kontainer basis data dihidupkan kembali menggunakan perintah `docker compose start db`. Begitu probe healthcheck berikutnya berhasil menjangkau PostgreSQL, Docker mendeteksi pemulihan dan secara otomatis mengembalikan status `uts-backend` menjadi `Up ... (healthy)` serta respons HTTP kembali 200 OK.

Rangkaian bukti visual pengujian dua arah ini disajikan secara komparatif pada Gambar 1.5.

```
❯ docker compose ps
NAME           IMAGE                    COMMAND                  SERVICE    CREATED          STATUS                    PORTS
uts-backend    uts-pl-543220-backend    "docker-php-entrypoi…"   backend    4 minutes ago    Up 4 minutes (healthy)    0.0.0.0:8000->8
000/tcp, [::]:8000->8000/tcp
uts-db         postgres:16-alpine       "docker-entrypoint.s…"   db         30 minutes ago   Up 3 minutes (healthy)    5432/tcp
uts-frontend   uts-pl-543220-frontend   "/docker-entrypoint.…"   frontend   6 minutes ago    Up 6 minutes              0.0.0.0:80->80/
tcp, [::]:80->80/tcp
```

(a) Kondisi Normal: Layanan Kontainer `uts-backend` Berstatus `(healthy)`

```
{
  "status": "unhealthy",
  "services": {
    "database": {
      "status": "down",
      "message": "Unable to connect to database.",
      "error": "SQLSTATE[08006] [7] could not translate host name \"db\" to address: Name does not resolve"
    },
    "cache": {
      "status": "down"
    }
  }
}
```

(b) Kondisi Gangguan: Basis Data Dimatikan, Status Kontainer Berubah Menjadi `(unhealthy)` dan Endpoint Membalas HTTP 503 Service Unavailable

```
❯ docker compose start db
[+] start 1/1
 ✔ Container uts-db Started                                                                                    0.2s
❯ docker compose ps
NAME           IMAGE                    COMMAND                  SERVICE    CREATED          STATUS                          PORTS
uts-backend    uts-pl-543220-backend    "docker-php-entrypoi…"   backend    9 minutes ago    Up 9 minutes (healthy)          0.0.0.
0:8000->8000/tcp, [::]:8000->8000/tcp
uts-db         postgres:16-alpine       "docker-entrypoint.s…"   db         36 minutes ago   Up 4 seconds (health: starting)   5432/t
cp
uts-frontend   uts-pl-543220-frontend   "/docker-entrypoint.…"   frontend   12 minutes ago   Up 12 minutes                   0.0.0.
0:80->80/tcp, [::]:80->80/tcp
```

(c) Kondisi Pemulihan: Basis Data Dihidupkan Kembali, Status Otomatis Pulih Menjadi `(healthy)`

Gambar 1.5: Tangkapan Layar Rangkaian Pembuktian Healthcheck Docker Compose Dua Arah (Sehat → Sakit → Pulih)

Gambar 1.5 menjadi bukti konklusif bahwa mekanisme *healthcheck* kontainer bekerja secara presisi dan sensitif terhadap fluktuasi dependensi infrastruktur riil.

Perubahan konfigurasi diabadikan melalui komit:

```
git commit -m "ci(docker): configure container healthcheck probe and
    outage resilience"
```

1.7 Integrasi Git, Pull Request, dan Otomasi CI/CD (Tahap 6)

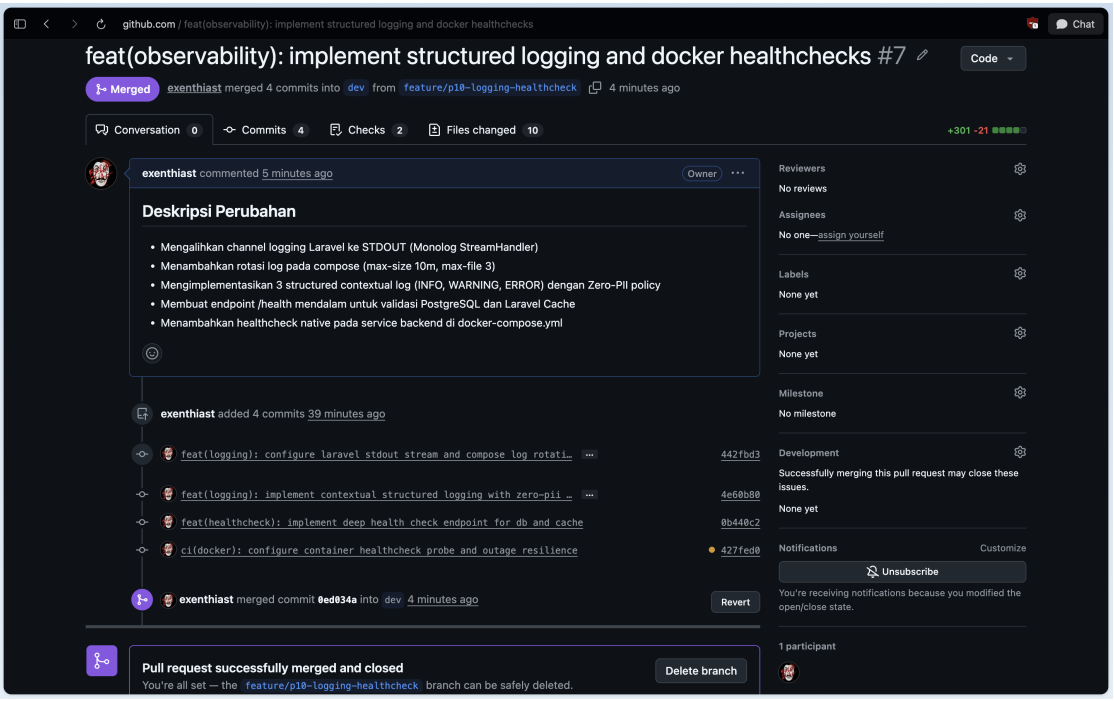
Setelah seluruh tahapan implementasi lokal selesai dan teruji sempurna, seluruh perubahan digabungkan ke cabang utama repositori melalui alur kerja *Pull Request* bertingkat.

1.7.1 Penggabungan Pull Request Bertingkat

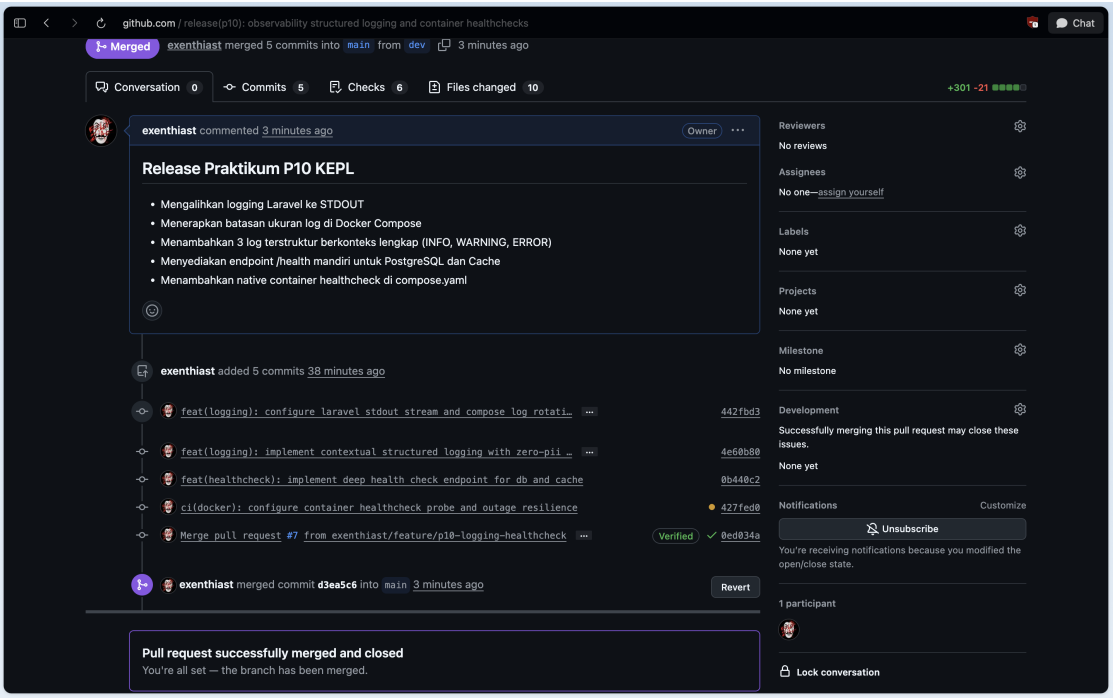
1. **Pull Request #7 (`feature/p10-logging-healthcheck` → `dev`):** Membawa 4 komit modular yang memuat konfigurasi logging stdout, rotasi log compose, implementasi 3 log terstruktur, endpoint health check mendalam, serta instrumen healthcheck kontainer. PR berhasil digabungkan ke cabang `dev`.

2. **Pull Request #8 (`dev` → `main`):** Merupakan rilis resmi fitur P10 yang menggabungkan seluruh pembaruan stabil ke cabang produksi utama `main`.

Tangkapan layar bukti penggabungan Pull Request #7 dan #8 disajikan pada Gambar 1.6.



(a) Bukti Penggabungan Pull Request #7 (`feature/p10-logging-healthcheck` ke `dev`)

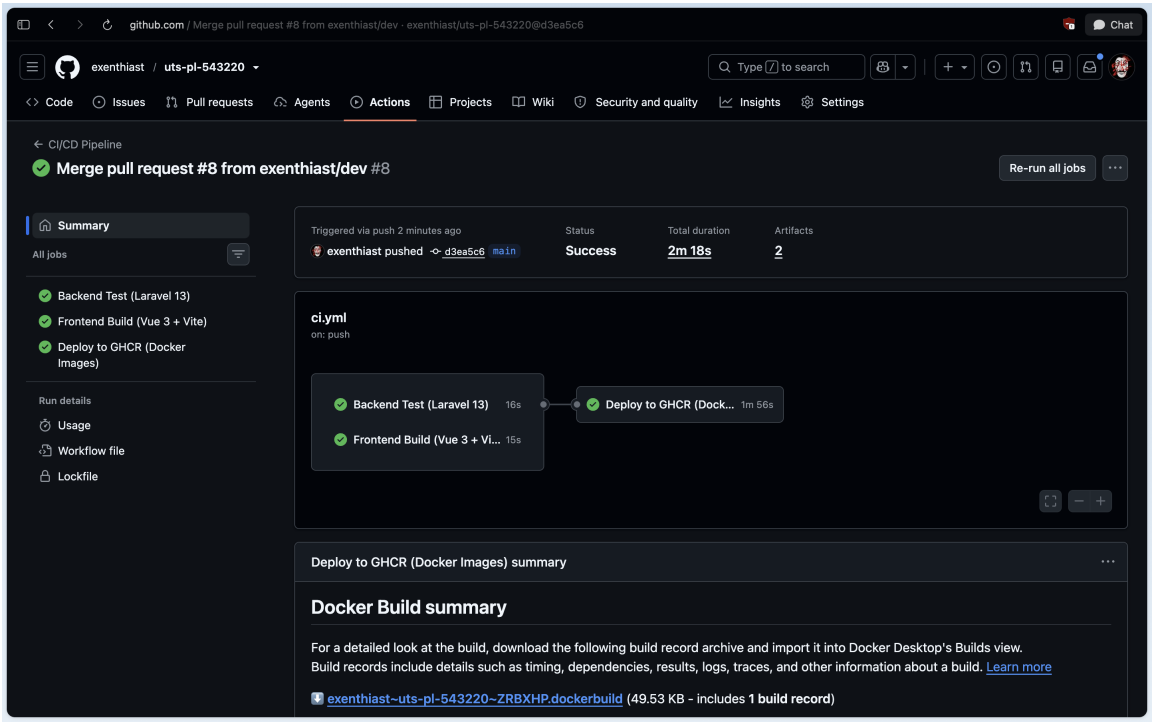


(b) Bukti Penggabungan Pull Request #8 (`dev` ke Cabang Rilis `main`)

Gambar 1.6: Tangkapan Layar Bukti Penggabungan Pull Request #7 dan #8 pada Repositori GitHub

1.7.2 Validasi Otomasi Pipeline GitHub Actions

Setiap pengajuan Pull Request dan dorongan komit ke cabang rilis memicu eksekusi alur kerja CI/CD pada berkas `.github/workflows/ci.yml`. Hasil eksekusi pipeline diperlihatkan pada Gambar 1.7.



Gambar 1.7: Tampilan Riwayat Eksekusi Pipeline Otomasi GitHub Actions yang Berjalan Sukses Hijau

Gambar 1.7 mengonfirmasi bahwa seluruh pekerjaan pengujian fitur backend (14 unit test), kompilasi frontend, serta pembangunan citra kontainer berhasil diselesaikan dengan status hijau sempurna (**All checks passed**).

1.8 Analisis Observabilitas Sistem dan Kesimpulan

Penerapan observabilitas modern melalui structured logging dan container healthchecks memberikan keunggulan teknis signifikan dalam siklus hidup perangkat lunak:

1. **Transparansi Operasional:** Pengalihan log ke `stdout` membebaskan aplikasi dari ketergantungan berkas lokal kontainer, memungkinkan integrasi instan dengan alat agregasi log kontainer Docker.

2. **Kemudahan Penelusuran Insiden (Auditability):** Log terstruktur dengan metadata aktor, aksi, dan penyebab mempercepat analisis akar masalah (*Root Cause Analysis*) tanpa membahayakan data rahasia pengguna.

3. **Ketahanan Mandiri (Self-Healing Readiness):** Healthcheck mendalam yang menguji basis data dan cache memungkinkan orkestrator kontainer (Docker Compose maupun Kubernetes) untuk mengambil keputusan failover, isolasi lalu lintas jaringan, atau restart otomatis kontainer yang tidak sehat.

1.9 Tautan Repositori dan Rekapitulasi Berkas Pengumpulan

Seluruh artefak kode sumber dan riwayat penggabungan telah dipublikasikan dan dirangkum pada Tabel 1.2.

Tabel 1.2: Rekapitulasi Luaran dan Tautan Pengumpulan Resmi Praktikum P10 KEPL

Komponen Pengumpulan	Keterangan / Tautan URL Resmi
Nama Repositori GitHub	`uts-pl-543220`
Tautan Repositori Utama	https://github.com/exenthiast/uts-pl-543220
Pemilik Repositori	`exenthiast` (Alfiz Desta Ariant Permana)
Cabang Fitur P10	`feature/p10-logging-healthcheck`
Cabang Rilis Utama	`main` dan `dev` (Terproteksi *Branch Protection*)
Tautan Pull Request #7	https://github.com/exenthiast/uts-pl-543220/pull/7
Tautan Pull Request #8	https://github.com/exenthiast/uts-pl-543220/pull/8
Berkas Endpoint Healthcheck	`backend/app/Http/Controllers/HealthController.php`
Berkas Konfigurasi Compose	`docker-compose.yml` (Healthcheck & Log Limits)
Peladen Aplikasi Backend	**FrankenPHP 1.4** (PHP 8.4 Alpine)
Perintah Eksekusi Sistem Lokal	`docker compose up -d -build`